Cahier des Charges : Système d'Optimisation des Tournées de Collecte

Département de Génie Informatique
Projet de Fin de Module

Année Universitaire 2025-2026

Table des matières

1 Introduction Générale **4**
1.1 Contexte du Problème 4
1.2 Objectif du Projet 4
1.3 Architecture Globale 4

2 Niveau 1 : Modèle de Base - Réseau Routier **5**
2.1 Objectif du Niveau 5
2.2 Modèle Mathématique 5
2.3 Classes à Implémenter 5
2.3.1 Classe `PointCollecte` 5
2.3.2 Classe `GrapheRoutier` 5
2.4 Fichiers d'Entrée/Sortie 6
2.4.1 Fichier d'Entrée : `input_niveau1.json` 6
2.4.2 Fichier de Sortie : `output_niveau1.json` 6
2.5 Tests de Validation 7

3 Niveau 2 : Affectation Camion ↔ Zone **7**
3.1 Objectif du Niveau 7
3.2 Modèle Mathématique 7
3.3 Classes à Implémenter 7
3.3.1 Classe `Camion` 7
3.3.2 Classe `Zone` 8
3.3.3 Classe `AffectateurBiparti` 8
3.4 Fichiers d'Entrée/Sortie 8
3.4.1 Fichier d'Entrée : `input_niveau2.json` 8
3.4.2 Fichier de Sortie : `output_niveau2.json` 9
3.5 Tests de Validation 10

4 Niveau 3 : Modèle Temporel Camion ↔ Zone ↔ Créneau **10**
4.1 Objectif du Niveau . . . 10
4.2 Modèle Mathématique . . . 10
4.3 Classes à Implémenter . . . 10
4.3.1 Classe `CreneauHoraire` . . . 10
4.3.2 Classe `ContrainteTemporelle` . . . 11
4.3.3 Classe `PlanificateurTriparti` . . . 11
4.4 Fichiers d'Entrée/Sortie . . . 11
4.4.1 Fichier d'Entrée : `input_niveau3.json` . . . 11
4.4.2 Fichier de Sortie : `output_niveau3.json` . . . 12
4.5 Tests de Validation . . . 13

5 Niveau 4 : VRP Enrichi avec Points Réels **13**
5.1 Objectif du Niveau . . . 13
5.2 Modèle Mathématique . . . 13
5.3 Classes à Implémenter . . . 13
5.3.1 Classe `Tournee` . . . 13
5.3.2 Classe `OptimiseurVRP` . . . 14
5.3.3 Classe `VisualiseurTournees` . . . 14
5.4 Fichiers d'Entrée/Sortie . . . 15
5.4.1 Fichier d'Entrée : `input_niveau4.json` . . . 15
5.4.2 Fichier de Sortie : `output_niveau4.json` . . . 15
5.5 Tests de Validation . . . 16

6 Niveau 5 : Système Dynamique Multi-Critères **16**
6.1 Objectif du Niveau . . . 16
6.2 Modèle Mathématique . . . 16
6.3 Classes à Implémenter . . . 17
6.3.1 Classe `CapteurIoT` . . . 17
6.3.2 Classe `SimulateurTempsReel` . . . 17
6.3.3 Classe `OptimiseurMultiObjectif` . . . 18
6.3.4 Classe `DashboardTempsReel` . . . 18
6.4 Fichiers d'Entrée/Sortie . . . 19
6.4.1 Fichier d'Entrée : `input_niveau5.json` . . . 19
6.4.2 Fichier de Sortie : `output_niveau5.json` . . . 20
6.5 Tests de Validation . . . 21

7 Stratégie de Développement et Livrables **21**
7.1 Arborescence du Projet . . . 21
7.2 Plan de Développement par Itérations . . . 23
7.3 Barème d'Évaluation . . . 23
7.4 Réutilisation entre Niveaux . . . 23

8 Annexe : Guide d'Implémentation **24**
8.1 Installation Environnement 24
8.2 Modèles de Données de Référence 24
8.3 Questions de Vérification par Niveau 24
8.3.1 Niveau 1 - Questions Fondamentales 24
8.3.2 Niveau 2 - Questions d'Affectation 25
8.3.3 Niveau 3 - Questions Temporelles 25
8.3.4 Niveau 4 - Questions d'Optimisation 25
8.3.5 Niveau 5 - Questions Dynamiques 25
8.4 Exemple d'Exécution Complète 25
8.5 Ressources et Références 26

1 Introduction Générale

1.1 Contexte du Problème

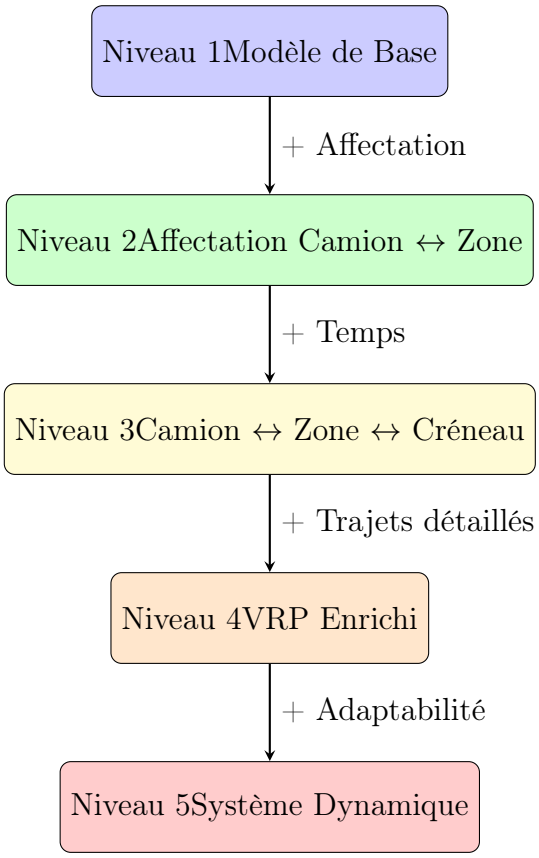
La ville de **VillePropre** souhaite optimiser son système de collecte des déchets ménagers. Actuellement, 5 camions parcourent la ville sans planification optimale, entraînant :

- Des surcoûts en carburant (environ 30% au-dessus des benchmarks)
- Une usure prématurée des véhicules
- Des retards de collecte dans certains quartiers
- Des plaintes citoyennes pour nuisances sonores

1.2 Objectif du Projet

Développer un système d'optimisation progressif en 5 niveaux, chaque niveau correspondant à un livrable fonctionnel indépendant mais intégré dans une architecture cohérente.

1.3 Architecture Globale



Progression des fonctionnalités :
→ Chaque niveau ajoute une complexité
→ Architecture modulaire
→ Tests indépendants par niveau

FIGURE 1 – Architecture en 5 niveaux du système

2 Niveau 1 : Modèle de Base - Réseau Routier

2.1 Objectif du Niveau

Calculer les distances optimales entre points de collecte en modélisant le réseau routier comme un graphe 1-partie.

2.2 Modèle Mathématique

- **Graphe** : $G = (V, E)$
- **Sommets** : $V = \{v_0, v_1, \ldots, v_n\}$ (dépôt + points de collecte)
- **Arêtes** : $E = \{(v_i, v_j) \mid \text{chemin direct existe}\}$
- **Pondération** : $w(v_i, v_j) = \text{distance euclidienne}(v_i, v_j)$

2.3 Classes à Implémenter

2.3.1 Classe `PointCollecte`

Listing 1 – Structure d'un point de collecte

```
class PointCollecte:
def __init__(self, id_point: int, x: float, y: float,
nom: str = ""):
self.id = id_point      # Identifiant unique
self.x = x              # Coordonnee X
self.y = y              # Coordonnee Y
self.nom = nom          # Nom du point (optionnel)

def distance_vers(self, autre_point) -> float:
"""Calcule la distance euclidienne vers un autre point"""
```

2.3.2 Classe `GrapheRoutier`

Listing 2 – Graphe du reseau routier

```
class GrapheRoutier:
def __init__(self):
self.sommets = {}        # Dict {id: PointCollecte}
self.aretes = {}         # Dict {(id1, id2): distance}

def ajouter_sommet(self, point: PointCollecte) -> None:
"""Ajoute un sommet au graphe"""

def ajouter_arete(self, id1: int, id2: int,
distance: float = None) -> None:
"""Ajoute une arete entre deux sommets"""

def plus_court_chemin(self, depart: int, arrivee: int) -> tuple:
"""Retourne (distance, chemin) avec Dijkstra"""
# Retour: (float, list[int])
```

```python

        def matrice_distances(self) -> list:
        """Retourne la matrice des distances entre tous les sommets"""
        # Retour: list[list[float]]
```

2.4 Fichiers d'Entrée/Sortie

2.4.1 Fichier d'Entrée : `input_niveau1.json`

Listing 3 – Format d'entree Niveau 1

```json
        {
            "depot": {
                "id": 0,
                "x": 0,
                "y": 0,
                "nom": "Centre de traitement"
            },
            "points_collecte": [
            {"id": 1, "x": 2.5, "y": 3.1, "nom": "Quartier Nord"},
            {"id": 2, "x": 5.2, "y": 4.8, "nom": "Centre Ville"},
            {"id": 3, "x": 7.8, "y": 1.2, "nom": "Zone Industrielle"}
            ],
            "connexions": [
            {"depart": 0, "arrivee": 1, "distance": null},
            {"depart": 1, "arrivee": 2, "distance": 4.2}
            ]
        }
```

2.4.2 Fichier de Sortie : `output_niveau1.json`

Listing 4 – Format de sortie Niveau 1

```json
        {
            "matrice_distances": [
            [0.0, 3.9, 6.8, 8.1],
            [3.9, 0.0, 4.2, 6.5],
            [6.8, 4.2, 0.0, 5.3],
            [8.1, 6.5, 5.3, 0.0]
            ],
            "chemins_calcules": [
            {
```

```
                    "depart": 0,
                    "arrivee": 3,
                    "distance": 8.1,
                    "chemin": [0, 1, 2, 3]
                }
                ]
            }
```

2.5 Tests de Validation

1. **Test 1.1** : Charger 10 points et vérifier que la matrice 10×10 est correcte
2. **Test 1.2** : Calculer le chemin 0→5 et vérifier distance > 0
3. **Test 1.3** : Vérifier symétrie : distance(i,j) = distance(j,i)
4. **Test 1.4** : Vérifier inégalité triangulaire

3 Niveau 2 : Affectation Camion ↔ Zone

3.1 Objectif du Niveau

Affecter optimalement chaque zone à un camion disponible en modélisant un graphe biparti.

3.2 Modèle Mathématique

- **Graphe biparti** : $G = (C \cup Z, E)$
- **Partie C** : $C = \{c_1, \ldots, c_m\}$ (camions)
- **Partie Z** : $Z = \{z_1, \ldots, z_n\}$ (zones)
- **Arêtes** : $E = \{(c_i, z_j) \mid \text{camion i peut servir zone j}\}$
- **Pondération** : $w(c_i, z_j) =$ cout estime de collecte

3.3 Classes à Implémenter

3.3.1 Classe `Camion`

Listing 5 – Structure d'un camion

```
class Camion:
def __init__(self, id_camion: int, capacite: float,
cout_fixe: float, zone_accessible: list = None):
self.id = id_camion
self.capacite = capacite        # en kg
self.cout_fixe = cout_fixe      # cout journalier
self.zone_accessible = zone_accessible or []
self.charge_actuelle = 0.0
```

3.3.2 Classe Zone

Listing 6 – Structure d'une zone

```python
class Zone:
def __init__(self, id_zone: int, points: list,
volume_estime: float, centre_x: float,
centre_y: float):
self.id = id_zone
self.points = points            # IDs des points
self.volume_estime = volume_estime
self.centre = (centre_x, centre_y)
```

3.3.3 Classe AffectateurBiparti

Listing 7 – Algorithme d'affectation

```python
class AffectateurBiparti:
def __init__(self, camions: list, zones: list,
graphe: GrapheRoutier):
self.camions = camions
self.zones = zones
self.graphe = graphe

def calculer_cout_affectation(self, camion_id: int,
zone_id: int) -> float:
"""Calcule le cout d'affectation d'un camion a une zone"""
# Retour: float

def affectation_gloutonne(self) -> dict:
"""Affectation gloutonne basee sur cout minimal"""
# Retour: Dict[int, list[int]] {camion_id: [zone_ids]}

def verifier_contraintes(self, affectation: dict) -> bool:
"""Verifie capacite et accessibilite"""
# Retour: bool

def equilibrage_charges(self, affectation: dict) -> dict:
"""Reequilibre les charges entre camions"""
# Retour: Dict[int, list[int]]
```

3.4 Fichiers d'Entrée/Sortie

3.4.1 Fichier d'Entrée : input_niveau2.json

```json
{
    "camions": [
    {
        "id": 1,
        "capacite": 5000,
```

```
                "cout_fixe": 200,
                "zones_accessibles": [1, 2, 3],
                "position_initial": {"x": 0, "y": 0}
            }
            ],
            "zones": [
            {
                "id": 1,
                "points": [1, 2, 3],
                "volume_moyen": 1200,
                "centre": {"x": 3.5, "y": 4.2},
                "frequence_collecte": "quotidien"
            }
            ],
            "contraintes": {
                "capacite_max_par_camion": 6000,
                "zones_incompatibles": [[1, 2]],
                "priorites": [{"zone_id": 1, "priorite": "haute
                  "}]
            }
        }
```

3.4.2 Fichier de Sortie : `output_niveau2.json`

```
        {
            "affectation": [
            {
                "camion_id": 1,
                "zones_affectees": [1, 3, 5],
                "charge_totale": 4500,
                "cout_estime": 320.5,
                "pourcentage_utilisation": 90
            }
            ],
            "statistiques": {
                "nombre_camions_utilises": 3,
                "charge_moyenne": 4200,
                "ecart_type_charge": 350,
                "zones_non_affectees": []
            }
        }
```

3.5 Tests de Validation

1. **Test 2.1** : Affecter 5 zones à 3 camions (capacités variées)
2. **Test 2.2** : Vérifier qu'aucun camion ne dépasse capacité
3. **Test 2.3** : Tester avec zone inaccessible → doit échouer
4. **Test 2.4** : Mesurer l'équilibrage : écart-type $< 20\%$ moyenne

4 Niveau 3 : Modèle Temporel Camion ↔ Zone ↔ Créneau

4.1 Objectif du Niveau

Planifier les créneaux horaires en modélisant un graphe triparti avec contraintes temporelles.

4.2 Modèle Mathématique

- **Graphe triparti** : $G = (C \cup Z \cup T, E)$
- **Partie C** : Camions
- **Partie Z** : Zones
- **Partie T** : $T = \{t_1, \ldots, t_k\}$ (créneaux temporels)
- **Arêtes** : (c_i, z_j, t_k) possible si :
  1. Camion c_i peut servir zone z_j
  2. Zone z_j accessible au créneau t_k
  3. Durée totale $\leq$ durée créneau

4.3 Classes à Implémenter

4.3.1 Classe `CreneauHoraire`

```
class CreneauHoraire:
def __init__(self, id_creneau: int, debut: str, fin: str,
jour: str, cout_congestion: float = 1.0):
self.id = id_creneau
self.debut = datetime.strptime(debut, "%H:%M")
self.fin = datetime.strptime(fin, "%H:%M")
self.jour = jour
self.cout_congestion = cout_congestion  # 1.0-2.0

def duree(self) -> float:
"""Retourne duree en heures"""
# Retour: float

def chevauche(self, autre_creneau) -> bool:
"""Verifie chevauchement"""
# Retour: bool
```

4.3.2 Classe ContrainteTemporelle

```python
class ContrainteTemporelle:
def __init__(self):
self.fenetres_zone = {}  # {zone_id: (debut, fin)}
self.pauses_camion = {}  # {camion_id: [(debut, fin)]}
self.congestion = {}     # {(zone_id, creneau): niveau}

def est_realisable(self, camion_id: int, zone_id: int,
creneau_id: int) -> bool:
"""Verifie faisabilite temporelle"""
# Retour: bool

def calculer_penalite(self, camion_id: int, zone_id: int,
creneau_id: int) -> float:
"""Calcule penalite temporelle (retard, congestion)"""
# Retour: float
```

4.3.3 Classe PlanificateurTriparti

```python
class PlanificateurTriparti:
def __init__(self, affectateur: AffectateurBiparti,
contraintes: ContrainteTemporelle):
self.affectateur = affectateur
self.contraintes = contraintes

def generer_plan_optimal(self, horizon_jours: int = 7) -> dict:
"""Genere plan hebdomadaire optimal"""
# Retour: Dict[str, list] {"lundi": [affectations]}

def resoudre_avec_contraintes(self) -> list:
"""Resout avec algorithme de programmation par contraintes"""
# Retour: list[tuple] [(camion_id, zone_id, creneau_id)]

def evaluer_plan(self, plan: dict) -> dict:
"""Evalue qualite du plan (respect horaires, cout, etc.)"""
# Retour: Dict[str, float]
```

4.4 Fichiers d'Entrée/Sortie

4.4.1 Fichier d'Entrée : input_niveau3.json

```json
{
    "creneaux": [
    {
        "id": 1,
        "debut": "08:00",
        "fin": "10:00",
        "jour": "lundi",
```

```
                "niveau_congestion": 1.2,
                "zones_exclusives": [3, 5]
            }
            ],
            "contraintes_temporelles": {
                "fenetres_zone": [
                {"zone_id": 1, "debut": "06:00", "fin": "18:00"}
                ],
                "pauses_obligatoires": [
                {"camion_id": 1, "debut": "12:00", "duree": 1}
                ],
                "zones_interdites_nuit": [1, 2, 3]
            },
            "horizon_planification": {
                "date_debut": "2024-01-15",
                "date_fin": "2024-01-21",
                "pas_temps": 30
            }
        }
```


4.4.2 Fichier de Sortie : `output_niveau3.json`


```
        {
            "planification_hebdomadaire": {
                "lundi": [
                {
                    "camion_id": 1,
                    "zone_id": 2,
                    "creneau": {"debut": "08:00", "fin": "10:00"},
                    "taches": [
                    {"point_id": 5, "ordre": 1, "heure_estimee": "08:30"},
                    {"point_id": 7, "ordre": 2, "heure_estimee": "09:15"}
                    ],
                    "duree_totale": 120,
                    "retard_estime": 0
                }
                ]
            },
            "indicateurs": {
                "taux_occupation": 85.3,
```

```
                    "respect_horaires": 92.1,
                    "congestion_moyenne": 1.15,
                    "retard_moyen": 4.2
                }
            }
```

4.5 Tests de Validation

1. **Test 3.1** : Planifier 3 camions sur 5 créneaux (10 zones)
2. **Test 3.2** : Vérifier non-chevauchenent pour même camion
3. **Test 3.3** : Tester fenêtre interdite → doit échouer
4. **Test 3.4** : Mesurer taux occupation > 70%

5 Niveau 4 : VRP Enrichi avec Points Réels

5.1 Objectif du Niveau

Générer des tournées concrètes avec optimisation des trajets individuels (Vehicle Routing Problem).

5.2 Modèle Mathématique

— **Graphe complet** : $G = (V, E)$ où $V = \text{dépôt} \cup P$
— **Objectif** : $\min \sum_k \sum_{i,j} c_{ij} x_{ijk}$
— **Contraintes** :
  1. $\sum_i d_i y_{ik} \leq Q_k$ (capacité)
  2. $\sum_k y_{ik} = 1$ (chaque point visité)
  3. Continuité de tournée

5.3 Classes à Implémenter

5.3.1 Classe `Tournee`

```
class Tournee:
def __init__(self, camion_id: int, points: list = None):
self.camion_id = camion_id
self.points = points or []  # liste ordonnee d'IDs
self.heure_depart = None
self.heure_retour = None

def ajouter_point(self, point_id: int, position: int = -1) -> bool
    :
"""Ajoute point a tournee avec verification capacite"""
# Retour: bool (succes/echec)
```

```python

        def calculer_distance(self, graphe: GrapheRoutier) -> float:
        """Calcule distance totale de la tournee"""
        # Retour: float

        def calculer_duree(self, temps_service: dict) -> float:
        """Calcule duree estimee avec temps de service"""
        # Retour: float

        def verifier_faisabilite(self, contraintes: dict) -> bool:
        """Verifie toutes les contraintes operationnelles"""
        # Retour: bool
```

5.3.2 Classe OptimiseurVRP

```python
        class OptimiseurVRP:
        def __init__(self, graphe: GrapheRoutier, camions: list):
        self.graphe = graphe
        self.camions = camions
        self.tournees = []

        def construire_solution_initiale(self) -> list:
        """Construction gloutonne (plus proche voisin)"""
        # Retour: list[Tournee]

        def algorithme_2opt(self, tournee: Tournee) -> Tournee:
        """Ameliore une tournee par echanges 2-opt"""
        # Retour: Tournee

        def recherche_tabou(self, iterations: int = 100) -> list:
        """Recherche Tabou pour optimisation globale"""
        # Retour: list[Tournee] (meilleure solution)

        def evaluer_solution(self, tournees: list) -> dict:
        """Evalue une solution complete"""
        # Retour: Dict[str, float]
```

5.3.3 Classe VisualiseurTournees

```python
        class VisualiseurTournees:
        def __init__(self, tournees: list, graphe: GrapheRoutier):
        self.tournees = tournees
        self.graphe = graphe

        def generer_carte(self, fichier_sortie: str) -> None:
        """Genere visualisation cartographique"""
        # Pas de retour, sauvegarde fichier

        def generer_gantt(self, fichier_sortie: str) -> None:
        """Genere diagramme de Gantt des tournees"""
        # Pas de retour, sauvegarde fichier
```

```python

        def calculer_statistiques(self) -> dict:
        """Calcule stats detaillees"""
        # Retour: Dict[str, any]
```

5.4 Fichiers d'Entrée/Sortie

5.4.1 Fichier d'Entrée : input_niveau4.json

```json
        {
            "configuration_vrp": {
                "nombre_camions": 3,
                "strategie_construction": "plus_proche_voisin",
                "methode_amelioration": "2opt",
                "critere_arret": {"iterations": 100, "stagnation": 10}
            },
            "points_detail": [
            {
                "id": 1,
                "type": "conteneur",
                "capacite": 120,
                "niveau_actuel": 85,
                "temps_service": 5,
                "fenetre": {"debut": "06:00", "fin": "20:00"},
                "restrictions": {"camion_mini": null, "acces": "facile"}
            }
            ],
            "parametres_optimisation": {
                "poids_distance": 1.0,
                "poids_temps": 1.5,
                "poids_equilibre": 0.8,
                "penalite_retard": 10.0
            }
        }
```

5.4.2 Fichier de Sortie : output_niveau4.json

```json
        {
            "tournees_optimisees": [
            {
                "camion_id": 1,
                "points_ordre": [0, 3, 7, 2, 5, 0],
```

```
                "distance_totale": 24.5,
                "duree_estimee": 185,
                "charge_finale": 4200,
                "heure_depart": "08:00",
                "heure_retour_estimee": "11:05",
                "couts": {"carburant": 18.5, "maintenance": 3.2
                    , "total": 21.7}
            }
            ],
            "performance": {
                "distance_totale": 78.3,
                "duree_moyenne_tournee": 172,
                "ecart_type_charge": 320,
                "taux_utilisation_camions": 88.5,
                "amelioration_vs_initial": 23.7
            },
            "visualisations": {
                "carte": "output/tournees_carte.png",
                "gantt": "output/gantt_tournees.png",
                "statistiques": "output/stats_detaillees.csv"
            }
        }
```


5.5 Tests de Validation

1. **Test 4.1** : Optimiser 20 points avec 3 camions
2. **Test 4.2** : Vérifier que 2-opt améliore la solution initiale
3. **Test 4.3** : Tester capacité dépassée → doit échouer
4. **Test 4.4** : Mesurer amélioration $> 15\%$ vs solution naïve

6 Niveau 5 : Système Dynamique Multi-Critères

6.1 Objectif du Niveau

Gérer dynamiquement la collecte avec réoptimisation en temps réel et objectifs multiples.

6.2 Modèle Mathématique

— **Graphe dynamique** : $G(t) = (V(t), E(t))$
— **Objectifs multiples** :

1. f_1 : Minimiser coût total
2. f_2 : Minimiser émissions CO_2

3. f_3 : Maximiser satisfaction citoyenne
4. f_4 : Équilibrer charge travail

— **Contraintes dynamiques** :
  — Remplissage aléatoire (capteurs IoT)
  — Trafic variable
  — Incidents (pannes, routes bloquées)

6.3 Classes à Implémenter

6.3.1 Classe `CapteurIoT`

```
class CapteurIoT:
def __init__(self, point_id: int, taux_remplissage: float = 0.0):
self.point_id = point_id
self.historique = []  # [(timestamp, niveau)]
self.seuil_alerte = 80.0

def lire_niveau(self) -> float:
"""Simule lecture capteur (aleatoire + tendance)"""
# Retour: float 0-100

def detecter_urgence(self) -> bool:
"""Detecte si besoin collecte urgente"""
# Retour: bool

def estimer_taux_remplissage(self, heures: int) -> float:
"""Predit niveau futur par regression lineaire"""
# Retour: float
```

6.3.2 Classe `SimulateurTempsReel`

```
class SimulateurTempsReel:
def __init__(self, optimiseur: OptimiseurVRP,
capteurs: dict, heure_debut: str = "08:00"):
self.optimiseur = optimiseur
self.capteurs = capteurs  # {point_id: CapteurIoT}
self.heure_actuelle = datetime.strptime(heure_debut, "%H:%M")
self.evenements = []
self.tournees_en_cours = []

def ajouter_evenement(self, type_event: str,
details: dict) -> None:
"""Ajoute evenement (panne, trafic, demande speciale)"""

def executer_pas_temps(self, duree_minutes: int = 15) -> dict:
"""Execute un pas de simulation"""
# Retour: Dict[str, any] (etat nouveau)

def replanifier_urgence(self, points_urgents: list) -> list:
"""Replanifie pour points urgents"""
```

```
        # Retour: list[Tournee]

        def calculer_emissions_co2(self, tournees: list) -> float:
        """Estime emissions CO2 (kg)"""
        # Retour: float
```

6.3.3 Classe OptimiseurMultiObjectif

```
        class OptimiseurMultiObjectif:
        def __init__(self, objectifs: list, poids: list = None):
        self.objectifs = objectifs  # liste fonctions objectif
        self.poids = poids or [1.0] * len(objectifs)
        self.solutions_pareto = []

        def evaluer_solution(self, tournees: list) -> list:
        """Evalue solution sur tous objectifs"""
        # Retour: list[float] (scores)

        def trouver_front_pareto(self, population: list,
        generations: int = 50) -> list:
        """Trouve front de Pareto par NSGA-II simplifie"""
        # Retour: list[tuple] (solutions non-dominees)

        def selectionner_solution(self, strategie: str = "equilibre") ->
            dict:
        """Selectionne solution selon strategie"""
        # Retour: Dict[str, any]
```

6.3.4 Classe DashboardTempsReel

```
        class DashboardTempsReel:
        def __init__(self, simulateur: SimulateurTempsReel):
        self.simulateur = simulateur
        self.metrics = {
            'co2_total': 0.0,
            'cout_total': 0.0,
            'satisfaction': 100.0,
            'points_urgents': []
        }

        def mettre_a_jour(self) -> None:
        """Met a jour tous les indicateurs"""

        def generer_rapport(self, periode: str = "journalier") -> dict:
        """Genere rapport detaille"""
        # Retour: Dict[str, any]

        def alerter_operateur(self, type_alerte: str,
        severite: int) -> None:
        """Genere alerte pour operateur"""
```

6.4 Fichiers d'Entrée/Sortie

6.4.1 Fichier d'Entrée : `input_niveau5.json`

```
{
    "simulation": {
        "duree_jours": 3,
        "pas_temps_minutes": 30,
        "heure_debut": "06:00",
        "heure_fin": "20:00",
        "generer_evenements": true
    },
    "capteurs_iot": [
    {
        "point_id": 1,
        "type": "ultrason",
        "precision": 95,
        "frequence_mise_a_jour": 5,
        "seuils": {"avertissement": 70, "urgence": 90}
    }
    ],
    "objectifs": [
    {"nom": "cout", "type": "minimiser", "poids": 0.3},
    {"nom": "co2", "type": "minimiser", "poids": 0.3},
    {"nom": "satisfaction", "type": "maximiser", "poids": 0.2},
    {"nom": "equite", "type": "maximiser", "poids": 0.2}
    ],
    "evenements_aleatoires": [
    {
        "type": "panne_camion",
        "probabilite": 0.05,
        "duree_moyenne": 120,
        "impact": "redistribution_points"
    },
    {
        "type": "trafic_intense",
        "probabilite": 0.15,
        "zone": [1, 3, 5],
        "impact": {"vitesse_reduite": 0.5}
    },
    {
        "type": "demande_speciale",
        "probabilite": 0.1,
```

```
                "exemples": ["dechets_verre", "encombrants"]
            }
            ]
        }
```

6.4.2 Fichier de Sortie : output_niveau5.json

```
        {
            "simulation_resultats": {
                "jour_1": {
                    "tournees_executees": 8,
                    "distance_totale": 156.7,
                    "emissions_co2": 47.0,
                    "couts_totaux": 1250.5,
                    "evenements": [
                    {"type": "panne", "camion": 2, "duree": 95, "impact": "mineur"},
                    {"type": "trafic", "zone": 3, "retard_moyen": 12}
                    ],
                    "satisfaction_moyenne": 88.5
                }
            },
            "front_pareto": [
            {
                "solution_id": 1,
                "scores": {"cout": 1200, "co2": 45, "satisfaction": 90, "equite": 85},
                "tournees": [...],
                "dominance": "non_dominee"
            }
            ],
            "recommandations": [
            {
                "type": "replanification",
                "justification": "3 points urgents detectes",
                "actions": ["redistribuer_points", "ajuster_horaires"],
                "gain_estime": 15.3
            },
            {
                "type": "maintenance_preventive",
                "camion": 3,
                "raison": "kilometrage_proche_limite",
```

```
                "urgence": "moyenne"
            }
            ],
            "dashboard": {
                "indicateurs_temps_reel": {
                    "co2_instantane": 2.3,
                    "couts_cumules": 850.2,
                    "taux_remplissage_moyen": 65.8,
                    "points_urgents_actuels": [7, 12, 25],
                    "camions_en_operation": 4,
                    "prochaines_collectes": [
                    {"point": 15, "heure_estimee": "14:30", "camion": 2}
                    ]
                },
                "alertes_actives": [
                {"type": "urgence", "point": 7, "niveau": 95, "depuis": "02:15"},
                {"type": "performance", "message": "Tournee 3 en retard de 18min"}
                ]
            }
        }
```


6.5 Tests de Validation

1. **Test 5.1** : Simuler 1 jour avec 5 événements aléatoires
2. **Test 5.2** : Détecter urgence quand remplissage $> 90\%$
3. **Test 5.3** : Vérifier replanification < 2 minutes
4. **Test 5.4** : Mesurer amélioration multi-objectif $> 10\%$

7 Stratégie de Développement et Livrables

7.1 Arborescence du Projet

```
projet_collecte_dechets/
|-- niveau1/                    # Livrable 1
|   |-- src/
|   |   |-- point_collecte.py
|   |   |-- graphe_routier.py
|   |   '-- main_niveau1.py
|   |-- data/
|   |   |-- input_niveau1.json
```

```
|   |   '-- output_niveau1.json
|   '-- tests/
|       '-- test_niveau1.py
|-- niveau2/                    # Livrable 2
|   |-- src/
|   |   |-- camion.py
|   |   |-- zone.py
|   |   |-- affectateur_biparti.py
|   |   '-- main_niveau2.py
|   |-- data/
|   |   |-- input_niveau2.json
|   |   '-- output_niveau2.json
|   '-- tests/
|       '-- test_niveau2.py
|-- niveau3/                    # Livrable 3
|   |-- src/
|   |   |-- creneau_horaire.py
|   |   |-- contrainte_temporelle.py
|   |   |-- planificateur_triparti.py
|   |   '-- main_niveau3.py
|   |-- data/
|   |   |-- input_niveau3.json
|   |   '-- output_niveau3.json
|   '-- tests/
|       '-- test_niveau3.py
|-- niveau4/                    # Livrable 4
|   |-- src/
|   |   |-- tournee.py
|   |   |-- optimiseur_vrp.py
|   |   |-- visualiseur_tournees.py
|   |   '-- main_niveau4.py
|   |-- data/
|   |   |-- input_niveau4.json
|   |   '-- output_niveau4.json
|   |-- tests/
|   |   '-- test_niveau4.py
|   '-- visualisations/         # Genere automatiquement
|-- niveau5/                    # Livrable 5
|   |-- src/
|   |   |-- capteur_iot.py
|   |   |-- simulateur_temps_reel.py
|   |   |-- optimiseur_multi_objectif.py
|   |   |-- dashboard_temps_reel.py
|   |   '-- main_niveau5.py
|   |-- data/
```

```
|   |   |-- input_niveau5.json
|   |   '-- output_niveau5.json
|   |-- tests/
|   |   '-- test_niveau5.py
|   '-- dashboards/             # Genere automatiquement
|-- commun/                     # Classes communes reutilisables
|   |-- outils_math.py
|   |-- parseur_json.py
|   |-- validateurs.py
|   '-- constantes.py
'-- README.md                   # Documentation complete
```

7.2 Plan de Développement par Itérations

Iteration	Duree	Livrable	Criteres d'Acceptation
1	1 semaine	Niveau 1	Tests 1.1 a 1.4 valides
2	1 semaine	Niveau 2	Tests 2.1 a 2.4 valides
3	1 semaine	Niveau 3	Tests 3.1 a 3.4 valides
4	2 semaines	Niveau 4	Tests 4.1 a 4.4 + visualisations
5	2 semaines	Niveau 5	Tests 5.1 a 5.4 + dashboard
6	1 semaine	Integration	Tous niveaux fonctionnent ensemble

Table 1 – Plan de développement iteratif

7.3 Barème d'Évaluation

Critere	Ponderation	Details
Fonctionnalite Niveau 1	10%	Calcul distances correct
Fonctionnalite Niveau 2	15%	Affectation optimale
Fonctionnalite Niveau 3	15%	Planification temporelle
Fonctionnalite Niveau 4	20%	Optimisation VRP
Fonctionnalite Niveau 5	20%	Systeme dynamique
Qualite du code	10%	PEP8, commentaires, docstrings
Tests unitaires	5%	Couverture $> 80\%$
Rapport technique	5%	Clarte, analyse resultats
Total	**100%**	

Table 2 – Bareme d'evaluation du projet

7.4 Réutilisation entre Niveaux

— **Niveau 1 → 2** : `GrapheRoutier` réutilisé pour calcul distances

— **Niveau 2 → 3** : `AffectateurBiparti` étendu avec temps
— **Niveau 3 → 4** : Contraintes temporelles intégrées dans VRP
— **Niveau 4 → 5** : `OptimiseurVRP` utilisé dans simulateur
— **Tous niveaux** : Classes `PointCollecte`, `Camion`

8 Annexe : Guide d'Implémentation

8.1 Installation Environnement

```
# 1. Creer environnement virtuel
python -m venv venv
source venv/bin/activate  # Linux/Mac
venv\Scripts\activate     # Windows

# 2. Installer dependances
pip install numpy matplotlib networkx pandas

# 3. Structure du projet
mkdir -p projet_collecte_dechets/{niveau{1..5}/{src,data,tests},
    commun}
```

8.2 Modèles de Données de Référence

Donnee	Type	Exemple
Point de collecte	Object	id, x, y, demande
Camion	Object	id, capacite, cout fixe
Zone	Object	id, points, volume
Creneau	Object	id, debut, fin, jour
Tournee	Object	camion, points ordonnes
Capteur IoT	Object	point, seuils, historique

Table 3 – Modeles de donnees communs

8.3 Questions de Vérification par Niveau

8.3.1 Niveau 1 - Questions Fondamentales

1. Le graphe est-il connexe ? (tous points accessibles)
2. Les distances sont-elles symétriques ?
3. Quelle complexité pour Dijkstra sur n points ?
4. Comment gérer les routes à sens unique ?

8.3.2 Niveau 2 - Questions d'Affectation

1. Quelle stratégie quand capacité insuffisante ?
2. Comment prioriser les zones importantes ?
3. Comment équilibrer charge entre camions ?
4. Que faire des zones non affectables ?

8.3.3 Niveau 3 - Questions Temporelles

1. Comment gérer les chevauchements ?
2. Quelle politique pour les retards ?
3. Comment optimiser les temps de trajet ?
4. Comment ajuster aux pics de trafic ?

8.3.4 Niveau 4 - Questions d'Optimisation

1. 2-opt améliore-t-il toujours ?
2. Quand arrêter l'optimisation ?
3. Comment visualiser efficacement ?
4. Quels indicateurs suivre ?

8.3.5 Niveau 5 - Questions Dynamiques

1. Quand déclencher replanification ?
2. Comment arbitrer entre objectifs ?
3. Comment gérer les incidents ?
4. Quelle fréquence de mise à jour ?

8.4 Exemple d'Exécution Complète

```python
# Exemple d'integration progressive
from commun.parseur_json import charger_configuration
from niveau1.graphe_routier import GrapheRoutier
from niveau2.affectateur_biparti import AffectateurBiparti
from niveau4.optimiseur_vrp import OptimiseurVRP
from niveau5.simulateur_temps_reel import SimulateurTempsReel

# 1. Charger configuration
config = charger_configuration("config_ville_propre.json")

# 2. Construire graphe de base
graphe = GrapheRoutier()
for point in config["points"]:
graphe.ajouter_sommet(point)

```

```python
        # 3. Affecter zones aux camions
        affectateur = AffectateurBiparti(config["camions"],
        config["zones"], graphe)
        affectation = affectateur.affectation_gloutonne()

        # 4. Optimiser tournees
        optimiseur = OptimiseurVRP(graphe, config["camions"])
        tournees = optimiseur.recherche_tabou(iterations=50)

        # 5. Simuler en temps reel
        simulateur = SimulateurTempsReel(optimiseur,
        config["capteurs"])
        resultats = simulateur.simuler_journee()

        print(f"Optimisation terminee. Gain total: {resultats['gain']}%")
```

8.5 Ressources et Références

- **Algorithmes** : Dijkstra, 2-opt, Recherche Tabou, NSGA-II
- **Librairies Python** : NetworkX (graphes), Matplotlib (visualisation), Pandas (analyse)
- **Références académiques** :
  1. Vehicle Routing Problem (VRP) - Toth & Vigo
  2. Multi-Objective Optimization - Deb et al.
  3. Dynamic Vehicle Routing - Psaraftis et al.
- **Données de test** : Fichiers JSON fournis + générateur aléatoire inclus